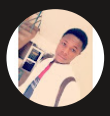
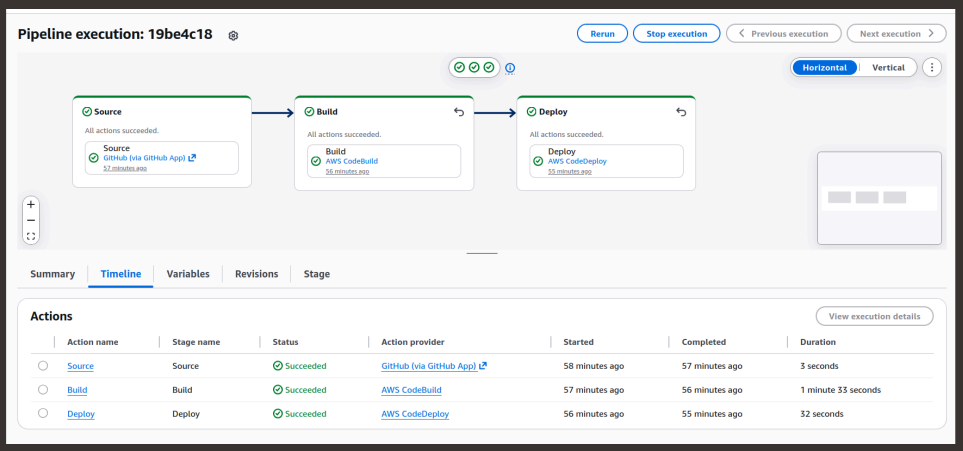
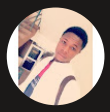
nextwork.org

Build a CI/CD Pipeline with AWS



Ahmed Tetteh





Ahmed Tetteh
NextWork Student

Introducing Today's Project!

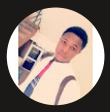
In this project, I will demonstrate CodePipeline's ability to automate my workflow. I'm doing this project to learn how CodePipeline can act as a central place to connect and oversee all my workflows, from building and running tests, to deployment. By the end of the project, I should be able to update the code for my webapp, push it and simply see all those updated changes in the live web app without manually going through CodeBuild and CodeDeploy.

Key tools and concepts

Services I used were AWS CodePipeline, CodeDeploy, CodeBuild, CodeArtifact, Amazon S3, GitHub, VS Code, IAM roles and policies, EC2 and CloudFormation. Key concepts I learnt include the different stages of the CI/CD pipeline, handling rollbacks and webhooks.

Project reflection

This project took me approximately 4 hours (including the startup and configuration of all other required resources, and error troubleshooting). The most challenging part was troubleshooting the errors that kept occured during my deployment stage and rollback. It was most rewarding to see all the green marker indicators on all the stages of the pipeline, indicating a successfull deployment of my application without going through the individual stages manually.



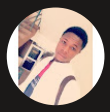
Ahmed Tetteh
NextWork Student

Starting a CI/CD Pipeline

AWS CodePipeline is a tool that automates all our developer processes (CI/CD pipeline), i.e, from building and testing to deployment. It makes our workflow seamless by detecting changes in our code when pushed to our Github repository, triggers a build process in CodeBuild while running tests simultaneously (Continous Integration), and then initiates a deployment of our application in CodeDeploy (Continuos Deployment). This significantly reduces our application deployment time for end-users.

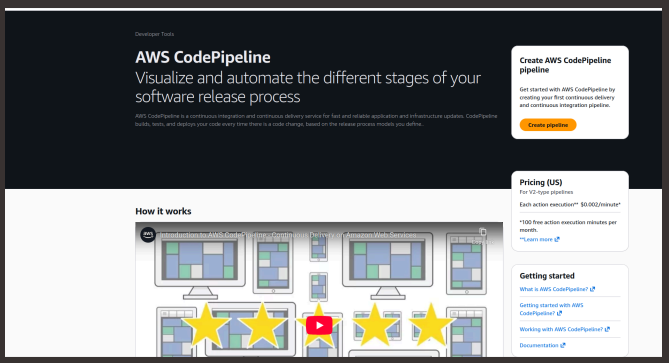
CodePipeline offers different execution modes based on how you want to handle multiple runs of your pipeline. I chose Superseded execution mode due to the size and nature of my project, but other options include Queued and Parallel mode.

A service role gets created automatically during setup so that CodePipeline can have the necessary permissions to perform actions on your behalf, i.e, store the build artifacts in your S3 bucket, initiate a build with CodeBuild and deploy your update with CodeDeploy.



Ahmed Tetteh
NextWork Student

nextwork.org



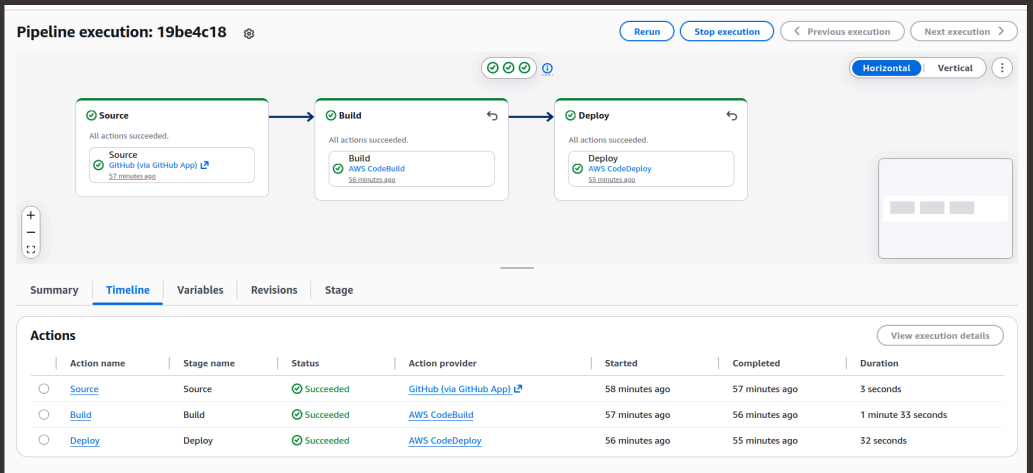
Ahmed Tetteh
NextWork Student

nextwork.org

CI/CD Stages

The three stages I've set up in my CI/CD pipeline are Source, Build and Deploy. While setting up each part, I learnt about how CodePipeline connects to each them, and also does it job, i.e, from using the CodeConnection to fetch and package the source code into zipped file(source artifact) > pass that source artifact from the S3 bucket to CodeBuild > deploy the build artifact using deployment group in CodeDeploy.

CodePipeline organizes the three stages into a single flow diagram from Source to Deploy. In each stage, you can see more details on the pipeline execution by clicking on the Stage ID. This also reveals hyperlinks for each action provider or exact resource CodePipeline used (GitHub, CodeBuild and CodeDeploy).

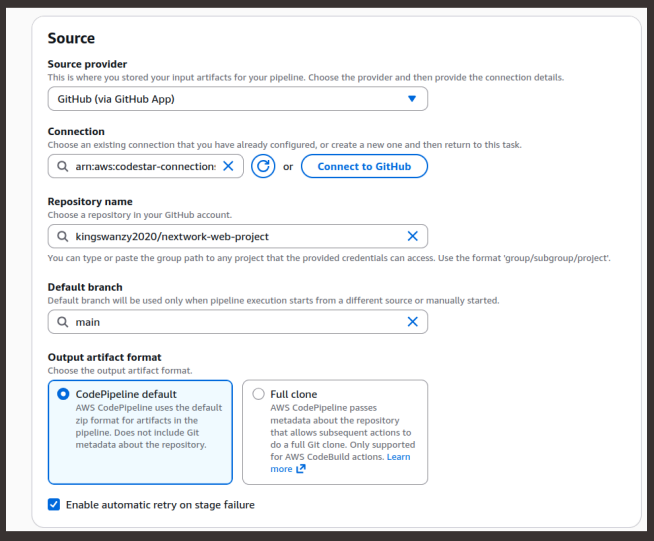


Ahmed Tetteh
NextWork Student

Source Stage

In the Source stage, the default branch tells CodePipeline which branch in our repository to look out for changes or updates to our source code. So, it's simply telling CodePipeline the bracnch to use to start a pipeline execution.

The source stage is also where you enable webhook events, which is the tool that initiates the automation of my pipeline. It's like a notification for CodePipeline to start a pipeline execution. It basically waits and listens for changes in the main/master branch (i.e, webhook/link to the GitHub repo) of our source repository and triggers the automation of our pipeline.

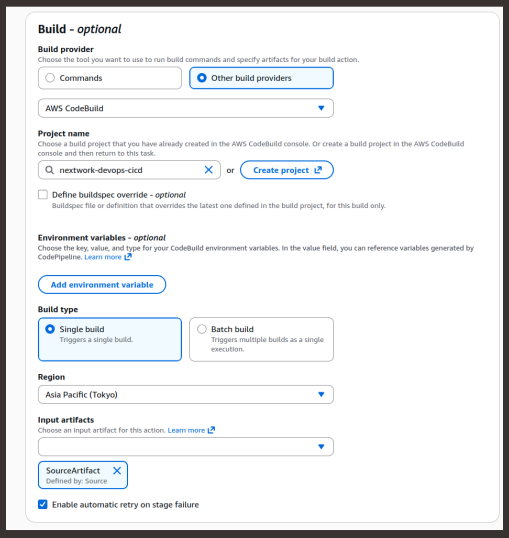


Ahmed Tetteh
NextWork Student

nextwork.org

Build Stage

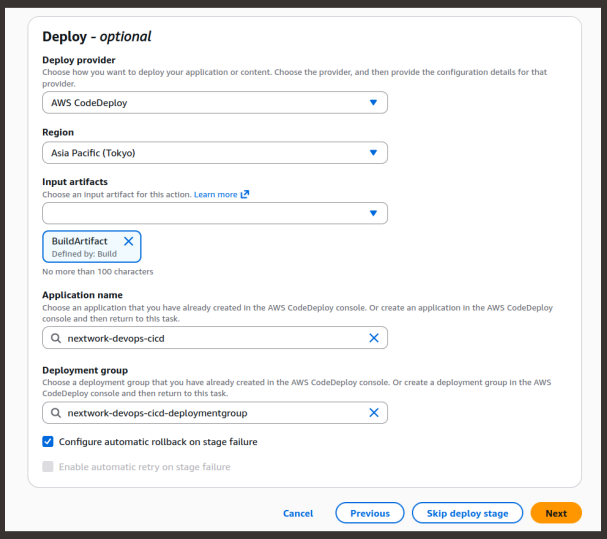
The Build stage sets up the connection to CodeBuild for our deployable. build artifact. I configured CodeBuild to use the source artifact as the input artifact of the build stage. The input artifact for the build stage is the compressed source code of the first stage in zipped file format.

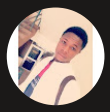


Ahmed Tetteh
NextWork Student

Deploy Stage

The Deploy stage is where deploy the end user product of your application, in my case, using CodeDeploy. I configured the build artifact as the input artifact for my deployment, along with it's application and deployment settings.





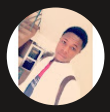
Ahmed Tetteh
NextWork Student

Success!

Since my CI/CD pipeline gets triggered by webhooks event whenever I push a change to the main branch of my GitHub repo, I tested my pipeline by making a change to the source code of my web app.

The moment I pushed the code change to GitHub, my pipeline was triggered by the webhooks event and did a new execution... The commit message under each stage reflects the latest trigger from GitHub for the pipeline execution. It also shows the latest change of the code.

Once my pipeline executed successfully, I checked my live web app to see verify the change on from the end-user perspective.



Ahmed Tetteh
NextWork Student

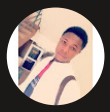
Hello King!

This is my NextWork web application working!

I am writing this line using nano instead of an IDE.

If you see this line in Github, that means your latest changes are getting pushed to your cloud repo :o

If you see this line, that means your latest changes are automatically deployed into production by CodePipeline!



Ahmed Tetteh
NextWork Student

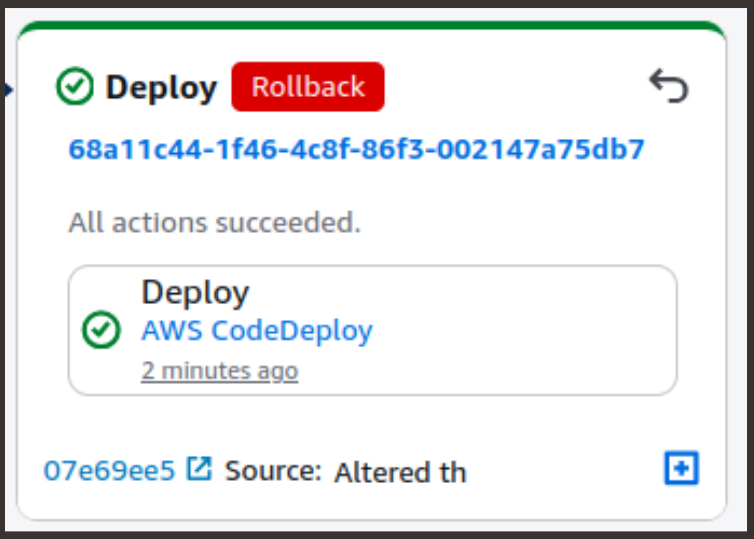
Testing the Pipeline

In a project extension, I initiated a rollback on the deploy stage to revert back to the previous version of my application, while maintaining the latest changes of my source and build stages. Automatic rollback is important for disaster recovery in cases where an update to the application has broken down some other features or introduced some bugs, hence, the application can be rolled back to the last known previous working version.

During the rollback, the source and build stage are unaffected by the current pipeline execution action because both stages come before the deployment stage for our rollback. I could verify this by comparing the commit messages related to each stage. I could see that both the Source and Build stage have the same commit message (i.e, they are using the same code change), while the Deploy stage has reverted back to the previous commit message.

After rollback, the live web app reverted to back to the previous state before the change in the code. This implies that, CodeDeploy was able to roll back the application to the previous version successfully, reducing application downtime and ensuring customer satisfaction.

Ahmed Tetteh
NextWork Student



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects